

Logical Reasoning Questions in C programming

Section 1: Output production:

① `int a = 5;`
`printf("%d", a++ + ++a);`

o/p: 12

`a++` → return 5, then `a=6`
`++a` → `a=7` return 7
→ post increment $\therefore 5 + 7 = 12$
→ pre increment

② `int a = 10;`

`if (a = 5)`
`printf("True");`
`else`
`printf("False");`

o/p: True

`(=)` is a assignment operator

if condition will check to equal or

not.

`5 = assigns (a)`

non-zero (true) → (5 is)

③ `int x = 10;`

`printf("%d", x == 10 ? 100 : 200);`

o/p: 100

In this `x = 10` assign the value and comparison to check condition $100 \overset{T}{=} 200 \rightarrow F$ so it is true, it will print

④ `int a = 5, b = 5;`

`if (a++ == ++b)`

`printf("Equal");`

`else`

`printf("Not equal");`

o/p: prints (Not Equal)

`++b` → pre increment returns 6.

`a++` → is post increment
 $\overset{5}{5} = 6$ is not equal.

⑤ `int x = 8;`

`x = x << 1;`

`printf("%d", x);`

o/p: 6

$\begin{matrix} 0011 \\ 0100 \end{matrix}$

return 6.

Right shift 1 time

⑨ `int arr[] = {1, 2, 3, 4};`
`printf("%d", arr[2]);`

o/p: 3.

arr have 4 elements
 print value of arr to
 count the value to arr+2
 0 index to 2 index, value is 3

⑩ `int x=0;`
`while (x < 3)`
`printf("%d", x++);`

o/p: 0 1 2.

$x=0$ is but in while loop
 have less than $0 < 3$ so
 it will ~~not~~ waiting returns 0, 1, 2
 3 time will continues.

⑪ `int x=2;`
`printf("%d", x++ * x++);`

o/p: 6

$x++$ return 2, $x++$ 3
 returns so $2 * 3 = 6$.

⑫ `int a=5;`
`int b=10;`
`printf("%d", a > b ? a : b);`

o/p: b=10 (prints).

Condition operator to
 check the value which is
 greater than so b is prints.
 $b=10$.

⑬ `char str[] = "Hello";`
`printf("%c", str);`

o/p: 'H'

In `str[] = "Hello"` so
 $\&str$ = points `str[0] = H`
 So they will print only
 the letter of H.

⑭ `int a=1;`
`int b=1;`
`if (a-- < b++)`
`printf("Yes");`

else

`printf("No");`

o/p: Yes.

$a--$ returns 0; $a=0$
 $b++$ returns 2; $b=2$
 so it will prints Yes.
 $a =$ return non-zero values $\rightarrow 1$
 1, 2, 1

⑬ int a=5;
printf("%d", size of (a++));

o/p: 4

int a=5
it all as size of (a++)
only consider size of
an integer or byte it will
print 4.

⑭ int x=1;
do {
printf("%d", x);
} while (x-- > 1);

o/p: 1

In this first will print 1
and use dowhile loop it can't
check < 0 > exit the loop
so prints 1.

⑮ int a=5;
int *p = &a;
*p = *p + 1;
printf("%d", a);

o/p: 6

In this, a=5 assign to
*p = &a so *p = 5 + 1 = 6
printf(a) so same value of
*p = &a a=6 print that 6 value.
*p + 1

Code Logic Understanding:

① int x=0;
if (x=10)
printf("Yes");

o/p: Yes

assign x=0.
if (x=0)
lets print 'Yes'.

② Const int x=10;
x=20;

o/p: error only one variable
to consider const variable
so

Conflicting type qualifiers for 'x'.

③ `int *ptr;`
`*ptr = 5;`

O/n: `*ptr` to assign the value `*ptr = 5.`

④ can a function return an array in C?

No, a function cannot return an array directly.

but there are common workarounds,

⇒ Return a pointer to a statically allocated array

⇒ pass array as an argument.

⇒ use dynamic memory allocation. (`malloc`).

⑤ `int a=5;`
`{ int a=10;`
`}`

O/n: Yes this declares to initialize an `variable` to accept redeclaration inner blocks and the inner declaration hides the outer one within its scope.

After the inner block ends, the outer `a` is still in scope and unchanged.

⑥ what happens if you access an array out of bounds?

Accessing an array out of bounds in C is undefined behaviour.

⇒ Crash

⇒ return garbage values

⇒ Corrupt memory

⇒ worst case

⇒ Cause a security vulnerability.

```
int main() {  
    int arr[3] = {1, 2, 3};  
    printf("%d\n", arr[5]);  
    return 0;  
}
```

Accessing out of bounds.

⑦ int i;
 for (i=0; i<10; i++)
 printf("%d", i);
o/p: 10

only it will print 0 to 9 only so give it ++ increment for counter.

⑧ int a=5;
 printf("%d", ++a);
o/p: error (value required
 a operand.

⑨ int a=1, b=2;
 int c = a++ + b;

q/n: Yes it's syntactically valid, though it looks confusing.

correct expression. $a++ + b$

⑩ Which statement is true about "static" keyword?

o/n: A static variable retain its value between function calls.

Static variable are allocated on the stack → False

" are re-initialized every time → False

" makes a function global → False.

```

⑪ int i=0;
while (i < 25)
    continue;
printf("val: 2");

```

o/p: 6

9a. How many i=0;

(0 25);

continue;

it will never to return

6.

⑫ what is 'NULL'?

NULL is a macro defined in `<stddef.h>` that represents a null pointer constant.

It means the pointer is not pointing to any valid memory address.

```

int *ptr = NULL;

```

⑬ which one is correct to declare a function returning pointer to int?

```

int * func();

```

```

int * func();

```

This means the function `func()` return a pointer to an int.

⑭ when does a segmentation fault occur in C?

A segmentation fault (segfault) occurs when a program tries to access.

→ Memory it's not allowed to (like dereferencing a NULL or dangling pointer);

→ Memory outside array bounds.

→ Freed memory.

```

int *ptr = NULL;

```

```

*ptr = 5;

```


(15) Can a void function return a value?

⇒ No, a function declared with void cannot return value.

⇒ It can have a return statement without a value to exist early.

(Void display c: {

printf("Hello\n");

return; // allowed but no value.

}

Section : 3 pointers and memory:

(① int a=5;

int *p = &a;

printf("i/d", *p);

o/p: 5

② int *ptr = NULL;

printf("i/d", *ptr);

o/p: dereferencing a NULL pointer, which leads to undefined behavior and most likely causes a segmentation fault.

NULL means the pointer doesn't point to a valid memory location. ptr, trying to access memory that your program doesn't own.

③ Diff: malloc () vs calloc ()

malloc ()

Memory allocation

malloc (n * size of (type))

does not initialization

slightly faster

when manual initialization preferred.

calloc ()

continuous memory allocation

calloc (n * size (type))

initialize to zero

slightly slower (due to zeroing)

when zeroed memory is needed.

④ what does 'free ()' do?

when pointer memory is free,
this free () function in C is used to deallocate dynamically allocated memory that was previously allocated using malloc (), calloc (), or realloc ().

void free (void *ptr);

⇒ ptr pointer to the memory block to be freed.

⇒ It does not return anything.

⇒ After free (ptr) the memory pointed by ptr is released back to the system.

⇒ it comes an dangling pointer:

(5) `int *p;`

`*p = 10;`

o/p: In this assigning an
`*p = 10` so, it will take address.

(6) can a pointer point to a function?

Yes, in C a pointer can point to a function. This is called pointer, and it is used to call functions indirectly, pass functions as arguments, or implement callbacks.

`return-type (*pointer-name) (parameter-type)`

(7) what is a dangling pointer?

A dangling pointer in C that does not point to a valid memory location - it 'declares' because the memory it used to point to it either, freed, deallocated, or out of scope, the pointer still holds old address.

```
int *ptr = malloc (size of(int));
```

```
*ptr = 10;
```

```
free (ptr);
```

```
*ptr = 20 => // undefined behaviour
```

```
free (ptr);
```

```
ptr = NULL;
```

Returning address of a local variable

```
int *getpointer ()
```

```
{  
    int x = 5;
```

```
    return &x;
```

```
}
```

Q How do you allocate a 2D array dynamically in C?

There are two common methods:

Method 1: using array of pointers:

```
int rows = 3, col = 4;
```

```
int **arr = malloc (rows * size of (int *));
```

```
for (int i = 0; i < rows; i++)
```

```
arr[i] = malloc (col * size of (int));
```

```
}
```

To free:

```
for (int i = 0; i < rows; i++)
```

```
free (arr[i]);
```

```
} free (arr);
```

Method 2:

using a single block

```
free (data);
```

```
free (arr);
```

Q What happens if you free() memory twice?

This is called double free, and it leads to undefined behaviour.

```
int *ptr = malloc (size of (int));
```

```
free (ptr);
```

```
free (ptr);
```

Safe

```
free (ptr);
```

```
ptr = NULL;
```



```

① int x = 10;
   int *p = &x;
   int *q = &p;
   printf("%d", *q);

```

o/p: 10

$x = 10$
 $*p = \&x \rightarrow$ 10 value x points to
 $\text{next } *q = \&p \rightarrow$ pointing to p
 So they output is 10.

Section: A Conceptual & Logical?

① Why is 'main()' special in C?

Reason:

Program entry point

OS Interface

Argument Support

Mandatory

Execution starts from main()

main() is return value signal
success/failure

can accept argc and argv

Every C program must have main()

② What is the output type of sizeof()?

sizeof() return value of type size_t.

size_t is an unsigned integer type, defined in <stddef.h> or <stdio.h>.

③ printf("%d", sizeof('A'));

'A' is a character constant, but in C, it's treated as an int (not char);

So sizeof('A') is size of (int) usually 4 bytes on most systems. o/p: 4

④ what are header files used for?

Header files (.h) declare functions macros, constants and data types.

Eg: #include <stdio.h> → printf() & scanf

#include <string.h> → strlen(), strcpy()

⑤ D/B: '=' and '==' in C

= Assignment operator a = 10

== Equality comparison if (a == 10)

⑥ what happens when break is used in a loop?

break; immediately exits the nearest enclosing loop

(for, while, do-while or switch).

```
for (int i = 0; i < 5; i++) {
```

```
    if (i == 2) break;
```

```
    printf("val", i);
```

```
}
```

⑦ what is a function pointer?

A pointer that stores the address of a function and can be used to call it.

```
int add (int x, int y) {return x+y;}
```

```
int (*fp) (int, int) = add;
```

```
printf("val", fp(2,3));
```


⑧ Q/b int func() and int func(void)

* int func () function takes unspecified arguments

* int func(void) function takes no arguments.

⇒ func() means it might accept arguments - not safe in modern C.

⇒ func(void) is more explicit and preferred.

⑨ what does volatile mean?

* volatile tells the compiler:

⇒ "Don't optimize this variable. Its value may change anytime".

used with:

Hardware registers, shared memory, Interrupts

Service routines.

"volatile int flags"

⑩ int a = 1;

int b = 0;

printf("id: a & b || a);

o/p: 0.

a & b → 1 & 0 → 0

!a → !1 → 0

0 || 0 → 0.